Cidades ESG Inteligentes

Atividade Avaliativa — Navegando pelo mundo DevOps

Aluno: Arthur Cavalcanti Granja

RM: 560650

Data: 21 de Abril de 2026

Repositório: https://github.com/artcgranja/DevOps-FIAP

Registry: ghcr.io/artcgranja/cidades-esg-api

Resumo

API REST em .NET 8 para gerenciar cidades e indicadores ESG (Environmental, Social, Governance). O projeto aplica práticas completas de DevOps: containerização multi-stage com Docker, orquestração via Docker Compose em dois ambientes isolados (staging e produção), e pipeline CI/CD automatizada no GitHub Actions que executa build, testes (xUnit), publica imagens no GitHub Container Registry e gera artefatos de deploy.

1. Descrição do Pipeline CI/CD

Ferramenta: GitHub Actions. O workflow está definido em `.github/workflows/ci-cd.yml`.

Gatilhos

- **Push em** `develop` → pipeline completo, publica imagem com tag `:staging`
- **Push em** `main` → pipeline completo, publica imagem com tag `:prod`
- **Pull Request em** `main` → apenas build + testes (não publica imagem)

Jobs (executados sequencialmente)

#	Job	O que faz
1	build-test	dotnet restore + dotnet build (Release) + dotnet test (xUnit, 6 testes) + upload dos resultados como artefato.
2	docker-build-push	Login no GHCR via GITHUB_TOKEN, build multi-stage da imagem Docker, push com tags :staging/:prod e :sha-<7> para rastreabilidade.
3	deploy	Determina o ambiente (main→prod, develop→staging), gera artefato 'deploy-instructions-<env>.txt' com comandos docker compose pull && up.

Lógica de rastreabilidade

Cada imagem publicada recebe **duas tags**: uma de ambiente (`:staging` ou `:prod`) e uma do commit SHA curto (`:sha-abc1234`). Isso permite promover, fazer rollback, e auditar exatamente qual commit gerou qual imagem em produção.

Observação sobre o deploy

Como o ambiente de execução final é a máquina local do aluno (requisito acadêmico, sem VPS), o job `deploy` não faz SSH para uma máquina remota — ele publica a imagem imutável no GHCR e entrega as instruções como artefato. A reprodução do deploy é 1 comando: `docker compose pull && up -d`.

2. Containerização com Docker

Arquitetura

Três ambientes totalmente isolados via arquivos de Docker Compose distintos. Cada ambiente possui sua própria rede bridge, volume persistente para o Postgres, portas mapeadas separadas e tag de imagem:

Ambiente	Compose	Porta API	Porta DB	Volume	Rede	Tag
Dev	docker-compose.yml	8080	5432	pg_data_dev	dev_net	build local
Staging	docker-compose.staging.yml	8081	5433	pg_data_staging	stg_net	:staging
Prod	docker-compose.prod.yml	8080	5432	pg_data_prod	prod_net	:prod

Dockerfile (multi-stage)

```
FROM mcr.microsoft.com/dotnet/sdk:8.0 AS build
WORKDIR /src
COPY *.sln ./
COPY src/CidadesEsg.Api/*.csproj src/CidadesEsg.Api/
RUN dotnet restore src/CidadesEsg.Api/CidadesEsg.Api.csproj
COPY src/CidadesEsg.Api/ src/CidadesEsg.Api/
RUN dotnet publish src/CidadesEsg.Api/CidadesEsg.Api.csproj \
    -c Release -o /app /p:UseAppHost=false

FROM mcr.microsoft.com/dotnet/aspnet:8.0 AS runtime
WORKDIR /app
COPY --from=build /app .
RUN apt-get update && apt-get install -y --no-install-recommends curl
EXPOSE 8080
ENV ASPNETCORE_URLS=http://+:8080
HEALTHCHECK --interval=30s --timeout=3s --retries=3 \
  CMD curl -f http://localhost:8080/health || exit 1
ENTRYPOINT ["dotnet", "CidadesEsg.Api.dll"]
```

Estratégias adotadas

• **Multi-stage:** o estágio `build` usa o SDK completo (~800MB); o estágio final `runtime` usa apenas o runtime ASP.NET (~110MB). Imagem final ~85% menor.
• **Ordem de COPY:** `.csproj` copiado antes do código-fonte — o cache do `dotnet restore` é reaproveitado quando só o código muda.
• **Healthcheck nativo:** Docker, Compose e Kubernetes detectam automaticamente container não-pronto e evitam rotear tráfego.
• **.dockerignore agressivo:** exclui `bin/obj`, `tests/`, `docs/`, `.git` — reduz o contexto enviado ao daemon.

Comandos principais

```
# Dev local
docker compose up -d --build

# Staging (pull da imagem do GHCR)
```

```
export GHCR_OWNER=artcgranja
docker login ghcr.io
docker compose -f docker-compose.staging.yml --env-file .env.staging up -d

# Prod (mesma lógica, compose diferente)
docker compose -f docker-compose.prod.yml --env-file .env.prod -p prod up -d

# Verificação
curl http://localhost:8081/health   # staging
curl http://localhost:8080/health   # prod
```

3. Evidências de Execução

3.1 Pipeline CI/CD rodando (GitHub Actions)

Página do GitHub Actions mostrando 5 execuções do workflow `CI/CD`, todas com status green (build + testes + push para GHCR + deploy instructions). As duas primeiras execuções correspondem à inauguração do pipeline em ambas as branches (`develop` → staging e `main` → prod).

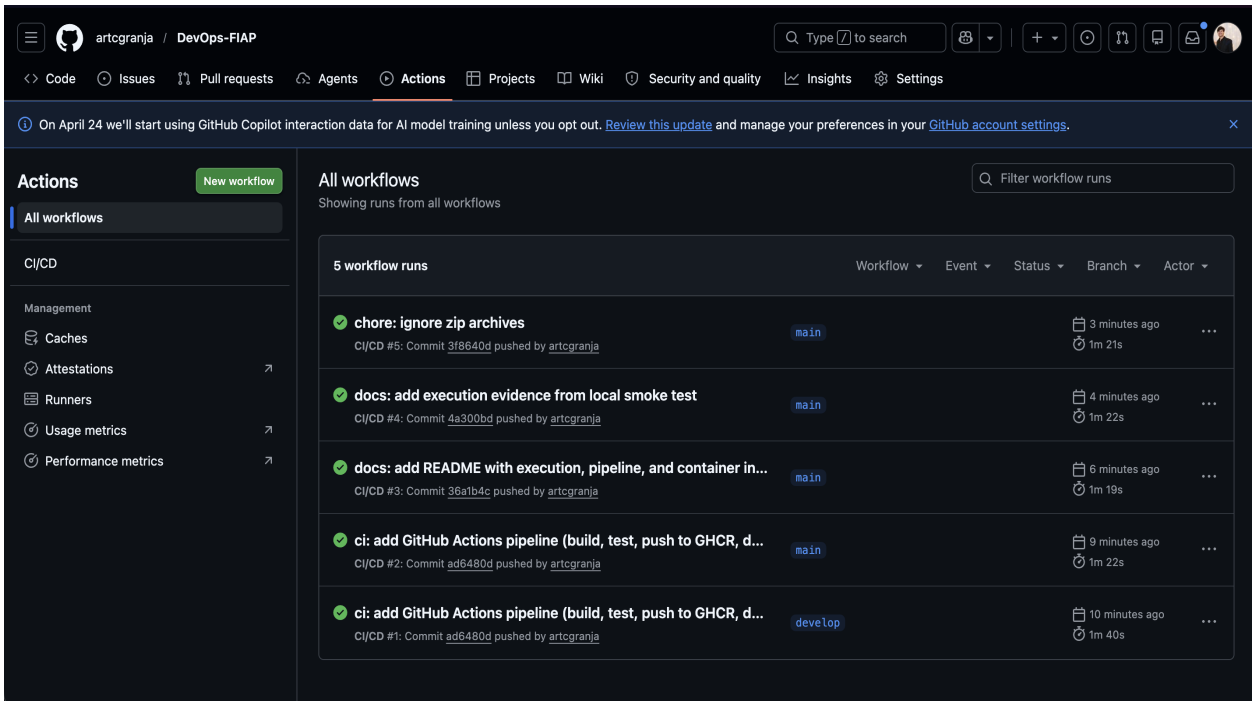

Figura 1 — GitHub Actions com 5 runs green. URL: github.com/artcgranja/DevOps-FIAP/actions

3.2 Ambientes staging e produção funcionando

Terminal mostrando (1) `docker ps` com os **4 containers rodando simultaneamente** — 2 APIs .NET (portas 8080 e 8081) + 2 PostgreSQL (portas 5432 e 5433), todos com status `healthy`; (2) `curl` nos endpoints `/health` de ambos os ambientes, retornando `{"status":"ok","db":"up"}`; (3) fluxo end-to-end: POST cria cidade em staging, GET retorna a lista atualizada.

```
~/github/devops-fiap git:(main) 0 • +0 -0 (0.072s)
docker ps --format "table {{.Names}}\t{{.Status}}\t{{.Ports}}"
NAMES               STATUS                      PORTS
prod-api-1          Up 55 seconds (healthy)     0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
prod-db-1           Up About a minute (healthy) 0.0.0.0:5432->5432/tcp, [::]:5432->5432/tcp
devops-fiap-api-1   Up About a minute (healthy) 0.0.0.0:8081->8080/tcp, [::]:8081->8080/tcp
devops-fiap-db-1    Up About a minute (healthy) 0.0.0.0:5433->5432/tcp, [::]:5433->5432/tcp

~/github/devops-fiap git:(main) 0 • +0 -0 (0.126s)
echo "=== STAGING (localhost:8081) ===" && curl -s http://localhost:8081/health && echo && echo "=== PROD (localhost:8080) ===" && curl -s http://localhost:8080/h
ealth && echo
=== STAGING (localhost:8081) ===
{"status":"ok","db":"up"}
=== PROD (localhost:8080) ===
{"status":"ok","db":"up"}

~/github/devops-fiap git:(main) 0 • +0 -0 (0.413s)
echo "--- POST cidade em staging ---" && curl -s -X POST http://localhost:8081/cidades -H "Content-Type: application/json" -d '{"nome":"São Paulo","estado":"SP"}'
 && echo && echo "--- GET cidades em staging ---" && curl -s http://localhost:8081/cidades && echo
--- POST cidade em staging ---
{"id":3,"nome":"São Paulo","estado":"SP","criadoEm":"2026-04-21T22:46:49.6168156Z","indicadores":[]}
--- GET cidades em staging ---
[{"id":1,"nome":"Curitiba","estado":"PR","criadoEm":"2026-04-21T22:39:31.713476Z","indicadores":[]},{"id":2,"nome":"Recife","estado":"PE","criadoEm":"2026-04-21T2
2:39:31.934438Z","indicadores":[]},{"id":3,"nome":"São Paulo","estado":"SP","criadoEm":"2026-04-21T22:46:49.616815Z","indicadores":[]}]
```

Figura 2 — docker ps + healthcheck staging/prod + fluxo CRUD.

4. Desafios Encontrados e Soluções

Instalação do .NET SDK sem sudo

Problema: O `brew install --cask dotnet-sdk` exige `sudo` via instalador .pkg. Sem acesso a terminal interativo, a instalação travava pedindo senha.

Solução: Usei o script oficial da Microsoft (`dotnet-install.sh`) que instala em `~/.dotnet` sem privilégios elevados. Adicionei ao `.zshrc` para persistir o PATH.

Ciclo de referência na serialização JSON

Problema: Ao retornar um `IndicadorEsg` após um POST, o EF Core já havia linkado a navegação bidirecional Cidade↔Indicadores. O `System.Text.Json` entrou em loop infinito e falhou com JsonException (cycle detected).

Solução: Configurei `ReferenceHandler.IgnoreCycles` globalmente via `ConfigureHttpJsonOptions`. Aproveitei e adicionei `JsonStringEnumConverter` para serializar o enum TipoIndicador como string legível.

Hook de segurança bloqueando o workflow do GitHub Actions

Problema: O hook de segurança avisou sobre interpolação direta de `${{ github.* }}` dentro de blocos `run:`, um vetor clássico de injeção em Actions quando o input vem de fonte não confiável.

Solução: Refatorei todas as expressões template para serem declaradas em `env:` do step e consumidas como variáveis de shell entre aspas duplas. Além de passar o hook, é a boa prática oficial recomendada pela própria documentação do GitHub.

Conflito de portas e nomes entre staging e prod

Problema: Rodando os dois `docker compose up` no mesmo host, o segundo falhava com conflito de nome de container (ambos usavam `devops-fiap-api-1`).

Solução: Usei a flag `-p prod` no compose de produção para forçar um project name distinto. Assim, staging fica sob o prefixo `devops-fiap-*` e prod sob `prod-*`, com redes e volumes totalmente isolados.

Platform mismatch arm64 × amd64

Problema: Imagens publicadas pelo GitHub Actions são amd64 (runner Ubuntu). Ao puxar no Mac M5 (arm64), o Docker alertava 'platform does not match'.

Solução: Deixei rodando via emulação (Rosetta/qemu do Docker Desktop) — é o comportamento correto para o escopo acadêmico. Em produção real, adicionaria `platforms: linux/amd64,linux/arm64` na action `docker/build-push-action` para publicar uma imagem multi-arch.

5. Checklist de Entrega

Item	Status	Onde conferir
Projeto compactado em .ZIP com estrutura organizada	✓	CidadesEsg-FIAP-entrega.zip (raiz do repo)
Dockerfile funcional	✓	Dockerfile (multi-stage, healthcheck, 110 MB)
docker-compose.yml ou arquivos Kubernetes	✓	docker-compose.yml + .staging.yml + .prod.yml
Pipeline com etapas de build, teste e deploy	✓	.github/workflows/ci-cd.yml (3 jobs)
README.md com instruções e prints	✓	README.md na raiz
Documentação técnica com evidências (PDF)	✓	Este documento
Deploy realizado nos ambientes staging e produção	✓	Figuras 1 e 2 + docs/evidencias/

6. Stack Técnica

Runtime: .NET 8 (SDK 8.0.420 / aspnet:8.0 na runtime image)

Framework API: ASP.NET Core Minimal API

ORM: Entity Framework Core 8

Banco de dados: PostgreSQL 16 (Alpine)

Testes: xUnit + WebApplicationFactory + SQLite in-memory (6 testes)

Containerização: Docker (multi-stage, healthcheck nativo)

Orquestração: Docker Compose v2 (3 arquivos: dev / staging / prod)

CI/CD: GitHub Actions (.github/workflows/ci-cd.yml)

Registry: GitHub Container Registry — ghcr.io

Controle de versão: Git + duas branches (develop → staging, main → prod)